

Train / Validation / Test Split

```
In [1]: import os
```

```
import pandas as pd
```

```
ROOT_PATH = os.path.dirname(os.getcwd())
```

```
DATA_FOLDER_PATH = os.path.join(ROOT_PATH, 'data', 'processed')
```

```
In [2]: df = pd.read_parquet(os.path.join(DATA_FOLDER_PATH, 'cleaned_arrivals.parquet'))
```

```
df.head()
```

```
Out[2]:
```

	access_route	state	origin_country	month	year	arrival_count	month_num	date	y
0	Aérea	Bahia	Afeganistão	Janeiro	2026	1.0	01	2026-01-31	
1	Aérea	São Paulo	Afeganistão	Janeiro	2026	4.0	01	2026-01-31	
2	Terrestre	Paraná	Afeganistão	Janeiro	2026	1.0	01	2026-01-31	
3	Terrestre	Rio Grande do Sul	Afeganistão	Janeiro	2026	1.0	01	2026-01-31	
4	Marítima	Rio de Janeiro	Afeganistão	Janeiro	2026	3.0	01	2026-01-31	

Split by Time

The monthly series is cut into three consecutive blocks:

Split	Months	Used for
train	everything before the validation year	fitting the models
validation	the <code>VAL_MONTHS</code> = 12 months before the test year	comparing and selecting models (notebooks 5 and 7)
test	the last <code>TEST_MONTHS</code> = 12 months of the data	the final evaluation only, in the final notebook

The test set is held out until the end: it is only read once, by the final notebook, after the model has been chosen on the validation year. The EDA (notebooks 3 and 4) and the feature engineering (notebook 6) see train + validation, never the test months.

The split dates are **derived from the data**, never typed by hand: when a new CSV is added and the notebooks are re-run, the three blocks roll forward on their own. Every later notebook reads its dates from `train.parquet`, `val.parquet` and `test.parquet`, so this cell is the single place that defines them.

Two checks guard the split:

- **No missing months.** The aggregated monthly series must be contiguous from the first to the last month. A gap would shift the 12-month windows.
- **Last month complete.** The ministry sometimes publishes partial months. If the last month is below 50% of the same month one year earlier, it is probably incomplete, and a warning is printed. Remove that month's rows (or wait for the full release) before modelling.

```
In [3]: VAL_MONTHS = 12
TEST_MONTHS = 12
INCOMPLETE_MONTH_RATIO = 0.5

monthly_totals = df.groupby('date')['arrival_count'].sum().sort_index()

expected_months = pd.date_range(
    monthly_totals.index.min(), monthly_totals.index.max(), freq='ME'
)
missing_months = expected_months.difference(monthly_totals.index)
assert missing_months.empty, f'Missing months in the data: {list(missing_months)}'

last_month = monthly_totals.index.max()
test_start = last_month - pd.offsets.MonthEnd(TEST_MONTHS - 1)
val_start = test_start - pd.offsets.MonthEnd(VAL_MONTHS)

print(
    f'Complete time range: {monthly_totals.index.min():%B %Y} to {last_month:%B %Y}'
    f'({len(monthly_totals)} months)'
)
val_end = test_start - pd.offsets.MonthEnd(1)
print(f'Validation window : {val_start:%B %Y} to {val_end:%B %Y}')
print(f'Test window      : {test_start:%B %Y} to {last_month:%B %Y}')

same_month_last_year = last_month - pd.offsets.MonthEnd(12)
ratio = monthly_totals[last_month] / monthly_totals[same_month_last_year]
print(
    f'Last month vs the same month one year earlier: {ratio:.2f} '
    f'({monthly_totals[last_month]:.0f} vs '
    f'{monthly_totals[same_month_last_year]:.0f})'
)
if ratio < INCOMPLETE_MONTH_RATIO:
    print(
        f'WARNING: {last_month:%B %Y} looks incomplete. Remove it from the raw CSV'
        'or wait for the full release before modelling.'
    )
```

Complete time range: January 1989 to August 2026 (452 months)
Validation window : September 2024 to August 2025
Test window : September 2025 to August 2026
Last month vs the same month one year earlier: 1.00 (576,276 vs 576,049)

```
In [4]: train_df = df[df['date'] < val_start]
print(
    f'Train set time range: {train_df["date"].min():%d %B %Y} to '
    f'{train_df["date"].max():%d %B %Y}'
)
train_df.tail()
```

Train set time range: 31 January 1989 to 31 August 2024

Out[4]:

	access_route	state	origin_country	month	year	arrival_count	month_num	d
952675	Fluvial	Rio Grande do Sul	Holanda	Agosto	2024	1.0	08	2008
952676	Fluvial	Rio Grande do Sul	Itália	Agosto	2024	1.0	08	2008
952677	Fluvial	Rio Grande do Sul	Paraguai	Agosto	2024	102.0	08	2008
952678	Fluvial	Rio Grande do Sul	Rússia	Agosto	2024	3.0	08	2008
952679	Fluvial	Rio Grande do Sul	Uruguai	Agosto	2024	2.0	08	2008

```
In [5]: val_df = df[(df['date'] >= val_start) & (df['date'] < test_start)]
assert val_df['date'].nunique() == VAL_MONTHS
print(
    f'Validation set time range: {val_df["date"].min():%d %B %Y} to '
    f'{val_df["date"].max():%d %B %Y}'
)
val_df.tail()
```

Validation set time range: 30 September 2024 to 31 August 2025

Out[5]:	access_route	state	origin_country	month	year	arrival_count	month_num	d
	976740	Aérea	Paraná	Índia	Agosto	2025	1.0	08 20 08
	976741	Aérea	Distrito Federal	Índia	Agosto	2025	11.0	08 20 08
	976742	Terrestre	Paraná	Índia	Agosto	2025	1.0	08 20 08
	976743	Terrestre	Paraná	Índia	Agosto	2025	31.0	08 20 08
	976744	Terrestre	Rio Grande do Sul	Índia	Agosto	2025	2.0	08 20 08

```
In [6]: test_df = df[df['date'] >= test_start]
assert test_df['date'].nunique() == TEST_MONTHS
print(
    f'Test set time range: {test_df["date"].min():%d %B %Y} to '
    f'{test_df["date"].max():%d %B %Y}'
)
test_df.tail()
```

Test set time range: 30 September 2025 to 31 August 2026

Out[6]:	access_route	state	origin_country	month	year	arrival_count	month_num
	985269	Terrestre	Rio Grande do Sul	Índia	Dezembro	2025	5.0 12
	985270	Marítima	Alagoas	Índia	Dezembro	2025	3.0 12
	985271	Marítima	Santa Catarina	Índia	Dezembro	2025	3.0 12
	985272	Marítima	Santa Catarina	Índia	Dezembro	2025	5.0 12
	985273	Fluvial	Amapá	Índia	Dezembro	2025	2.0 12

Output

```
In [7]: # the three blocks must be consecutive and cover every row exactly once
assert len(train_df) + len(val_df) + len(test_df) == len(df)
assert val_df['date'].min() == train_df['date'].max() + pd.offsets.MonthEnd(1)
assert test_df['date'].min() == val_df['date'].max() + pd.offsets.MonthEnd(1)

train_df.to_parquet(os.path.join(DATA_FOLDER_PATH, 'train.parquet'), index=False)
```

```
val_df.to_parquet(os.path.join(DATA_FOLDER_PATH, 'val.parquet'), index=False)
test_df.to_parquet(os.path.join(DATA_FOLDER_PATH, 'test.parquet'), index=False)
```